TIPOS DE DATOS

MAPAS

Un mapa es una colección de tuplas **clave-valor**.
La ***clave*** solo puede ***aparecer una vez*** en la colección.
Los datos se almacenan en el orden de inserción.
No utiliza ***índices*** para acceder a sus elementos

CREACIÓN DE UN MAPA:

```
const mapa = new Map();
```

```
const agenda = new Map([
  [555,"Pía"], [556, "Pol"]
]);
```

RECORRIDO DE UN MAPA:

```
for (persona of agenda){
  console.log(persona);
}
```

```
▸ (2) [555, 'Pía']
▸ (2) [556, 'Pol']
```

```
for (let numero of agenda.keys()){
  console.log(numero);
}
```

```
555
556
```

```
for (let[numero,nombre] of agenda){
  console.log(`${nombre} tiene el telefono ${numero}`);
}
```

```
Pía tiene el telefono 555
Pol tiene el telefono 556
```

```
for (let nombre of agenda.values()){
  console.log(nombre);
}
```

```
Pía
Pol
```

TIPOS DE DATOS

MAPAS

```
const agenda = new Map([
  [555,"Pía"], [556, "Pol"]
]);
```

ADICIÓN DE ELEMENTOS A UN MAPA:

```
agenda.set(554,"Tom");
```

```
Pía tiene el telefono 555
Pol tiene el telefono 556
Tom tiene el telefono 554
```

```
agenda.set(554,"Tom").set(553,"Bill");
```

```
Pía tiene el telefono 555
Pol tiene el telefono 556
Tom tiene el telefono 554
Bill tiene el telefono 553
```

```
agenda.set(554,"Tom");
agenda.set(554,"Tom");
agenda.set(554,"Tom");
```

```
Pía tiene el telefono 555
Pol tiene el telefono 556
Tom tiene el telefono 554
```

BORRAR ELEMENTOS

```
agenda.delete(556);
```

FUNCIONES DE MAPAS

```
if (agenda.has(556)){    }  --> Devuelve true si 556 está en agenda

agenda.get(553) --> valor de la clave 553

[...agenda]   --> Convierte el mapa en array de arrays
```

...
operador arrastre, propagación o spread

RECORRIDO: forEach()

forEach permite configurar un recorrido de arrays, conjuntos o mapas usando una función que recibe dos parámetros: el primero almacena automáticamente cada elemento y el segundo (opcional) su índice. Ignora los elementos "undefined".

```
let array = ["a","bol","b","agua"];
array.forEach(function(elemento,i){
  console.log(`Posición ${i}: ${elemento}`);
});
```

```
Posición 0: a
Posición 1: bol
Posición 2: b
Posición 3: agua
```

```
let conjunto = new Set();
conjunto.add("A").add("B").add("C");
conjunto.forEach(function(elemento){
  console.log(`Elemento: ${elemento}`);
});
```

```
Elemento: A
Elemento: B
Elemento: C
```

```
let mapa = new Map();
mapa.set("a",27).set("b",90).set("c",87);
mapa.forEach(function(valor,clave){
  console.log(`Clave:${clave} - Valor: ${valor}`);
});
```

```
Clave:a - Valor: 27
Clave:b - Valor: 90
Clave:c - Valor: 87
```

MÁS METODOS DE ARRAYS: sort()

El método ***sort***() ordena un array considerando la posición que ocupa cada carácter en la tabla ***Unicode***. Pero puede incluir una función ***callback*** para establecer un criterio de ordenación personalizado. La función debe recibir dos parámetros, de manera que si la función devuelve:

< 0 : se sitúa primer parámetro antes
= 0 : no se realizan cambios
> 0 : se sitúa el segundo parámetro antes

```
let array = ["a","bol","b","agua"];

array.sort();
  -->▸ (4) ['a', 'agua', 'b', 'bol']

array.sort((primero,segundo) => primero.length - segundo.length));
  -->▸ (4) ['a', 'b', 'bol', 'agua']
```

MÁS METODOS DE ARRAYS: map()

map() es una función para array establecida mediante una ***callback con un parámetro***. La función map() no modifica el array inicial, y devuelve ***otro*** array al que se ha aplicado la función callback a cada elemento del array.

```
let sinIva = [15, 1, 30.5];
let conIva = sinIva.map((x) => x * 1.21);
  -->  ▸ (3) [18.15, 1.21, 36.905]
```

MÁS METODOS DE ARRAYS: filter()

filter() es una función para array establecida también mediante una ***callback con un parámetro*** que va recogiendo el valor de cada elemento del array. En cada iteración comprueba si el elemento cumple una función específica. La función filter() no modifica el array inicial, y devuelve ***otro*** array con aquellos elementos que cumplen la condición.

```
let nombres = ["Eva", "Elsa", "Julia"];
let nombresE = nombres.filter((x) => x.startsWith("E"));
  -->
    ▸ (2) ['Eva', 'Elsa']
```

HTML5 proporciona varios atributos que describen un evento. En el HTML debemos añadir a un elemento el atributo correspondiente con el código que queremos que se ejecute.

ATRIBUTOS ASOCIADOS AL MOVIMIENTO DEL RATÓN

- ***onclick*** - usuario hace clic con el botón izquierdo del ratón.
- ***ondblclick*** - usuario hace doble clic con el botón izquierdo del ratón .
- ***oncontextmenu*** - usuario hace clic con el botón derecho del ratón.
- ***onmousedown*** - usuario pulsa el botón izquierdo o el derecho del ratón.
- ***onmouseup***- usuario libera el botón izquierdo del ratón.
- ***onmouseenter*** – el ratón se introduce en el área ocupada por el elemento.
- ***onmouseleave*** – el ratón abandona el área ocupada por el elemento.
- ***onmouseover*** – el ratón se mueve sobre el elemento o cualquiera de sus hijos.
- ***onmouseout*** – el ratón abandona el área ocupada por el elemento o cualquiera de sus hijos.
- ***onmousemove*** – el ratón se encuentra sobre el elemento y se mueve.
- ***onwheel*** – se hace girar la rueda del ratón.

```
<input type="button" value="Adelante" id="boton" onclick="jugar()">
```

ATRIBUTOS ASOCIADOS A EVENTOS GENERADOS POR EL TECLADO

- ***onkeypress*** - se activa el elemento de entrada y se pulsa una tecla.
- ***onkeydown*** - se activa el elemento de entrada y se pulsa una tecla.
- ***onkeyup*** - se activa el elemento de entrada y se libera una tecla.

Este tipo de atributos se aplican a elementos que aceptan una entrada del usuario, como <input> o <textarea>.

```
<input type="text" name="dni" id="dni" onkeypress="alert('Utilice May.')">
```

ATRIBUTO ASOCIADO AL DOCUMENTO

- ***onload*** - un recurso termina de cargarse

Este atributo puede aplicarse, por ejemplo, al body.

```
<body onload="alert('Bienvenido!')">
```

Excepciones o errores de tiempo de ejecución.
De forma predeterminada, *las excepciones* lanzadas interrumpen la ejecución del programa y hacen que la información correspondiente aparezca en la consola.

```
> let multiply = (a b) => a + b;
✕ Uncaught SyntaxError: Unexpected identifier 'b'
```

```
> let producto = (a,b) => a+b;
<· undefined
> let resultado = produto (10,20);
✕ ▶Uncaught ReferenceError: produto is not defined
```

Situaciones erróneas que no generan excepciones.
la información sobre el error es el valor específico que se devuelve

```
console.log(100 / 0);                // -> Infinity
console.log(100 * "2");              // -> 200
console.log(100 * "abc");            // -> NaN
console.log(Math.pow("abc", "def")); // -> NaN
```

```
> let producto = (a, b) => a + b;
  let result = producto(10, 20);
  console.log(result);
  30
```

Error lógico.
El resultado obtenido es incorrecto

*nombre de la variable que contendrá información sobre el error detectado. Puede tener cualquier nombre válido, pero el nombre **error** es el de uso más extendido.*

```
try {
   // codigo      riesgoso
} catch (error) {
   // bloque opcional
   // Se ejecuta si se lanza una excepción en el try
} finally {
   // bloque opcional
   // se ejecuta haya o no excepción, se ejecute o no catch
}
```

Los bloques ***catch*** y ***finally*** son opcionales, pero debe existir al menos uno de los dos.

- Si el código encerrado en el bloque ***try*** lanza una ***excepción***, se ejecutará el código del bloque ***catch*** (si existe).

  Si no se genera **ninguna excepción**, el código del bloque ***catch se ignora***.
- Si hay bloque ***finally***, se ***ejecutará siempre***, independientemente de que se haya producido una excepción o no.
- Si no hay bloque finally, después de ejecutar el bloque catch, el programa continuará ejecutándose en la siguiente instrucción.

```
try {
   // codigo     riesgoso
} catch (error) {
   // Se ejecuta si ocurre una excepción en el try
   if (error instanceof ReferenceError) {
     // Tratamiento del tipo de error
   }
   else if (error instanceof TypeError){
     // Tratamiento del tipo de error
   }
   else {
     // Tratamiento resto errores
   }

} finally {
   // bloque de ejecución obligada
}
```

El operador ***instanceof*** nos permite reaccionar de manera diferente según el ***tipo específico de error*** ocurrido:

SintaxError, ReferenceError , TypeError, RangeError

EvalError, InternalError, URIError

Si el código del bloque ***catch*** lanza una excepción, esta se puede recoger en el bloque finally.

```
let a = 10;
try {
    a = b;  // ReferenceError
} catch (error) {
    console.log(`Valor desconocido de ${b}`);
                  // -> ReferenceError
} finally {
    console.log("Ups, ha habido errores");
}
```

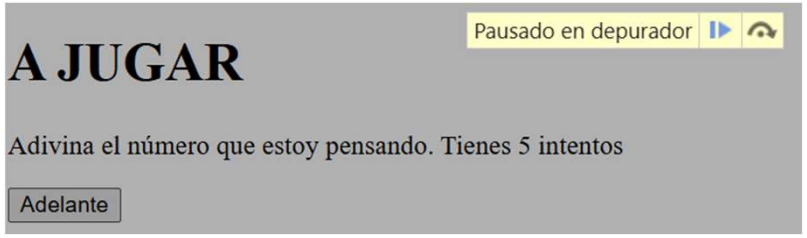
```javascript
let a = 10;
try {
    a = b; // Primer ReferenceError
} catch (error) {
    try {
        console.log(b); // Segundo ReferenceError
    } catch {
        console.log("Segundo catch!");
    }
} finally {
    console.log("Finally!");
}
```

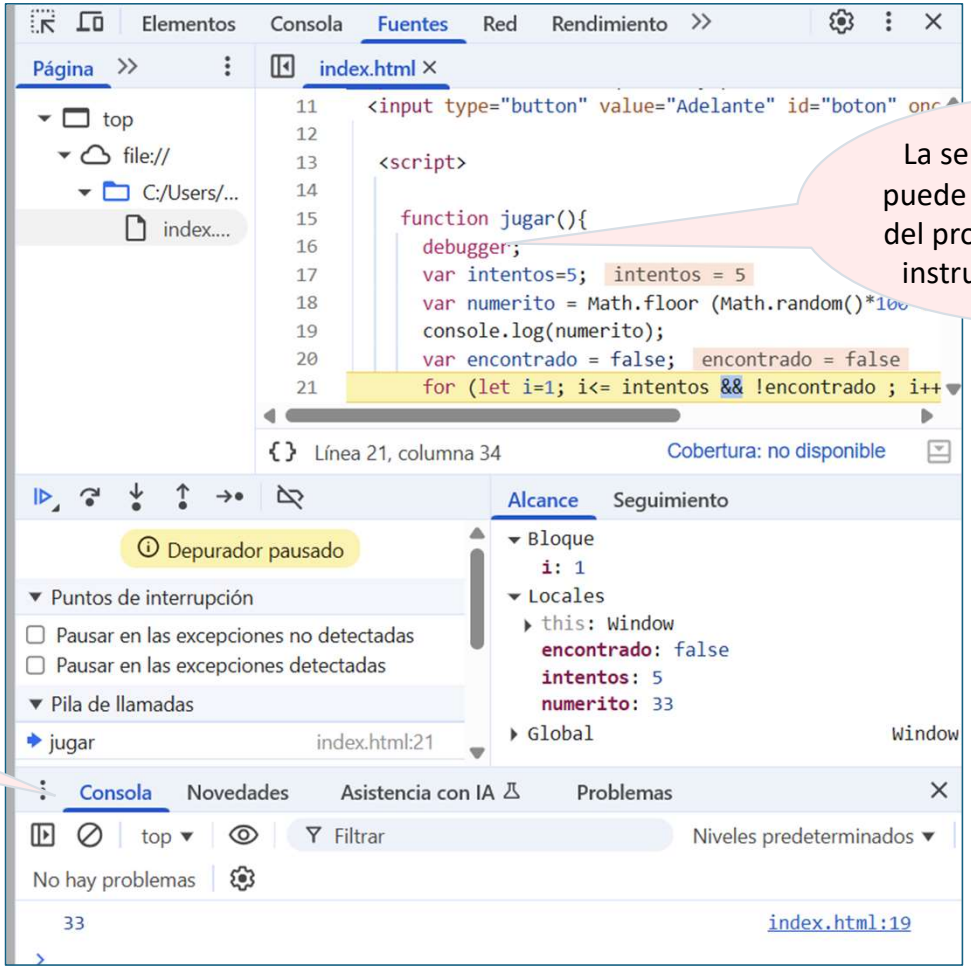
Los bloques ***try-catch -finally*** se pueden anidar.

La instrucción ***throw*** lanza manualmente una excepción. Le sigue cualquier valor que se considere una excepción (un número o uno de los objetos de error predefinidos)

```javascript
throw 100; // -> Uncaught 100
throw new ReferenceError("Variable no declarada");
```

La instrucción ***throw detendrá la ejecución*** a menos que se recoja dentro de un try-catch





La sentencia ***debugger*** puede detener la ejecución del programa y reanudarla, instrucción a instrucción

Con el ***debugger*** activado se puede usar la consola. ***ESC*** la oculta o la muestra

Elementos | Consola | Fuentes | Red | Rendimiento | Memoria | Aplicación

Página | index.html

top
file://
C:/Users/...
index....

```
14  <script>
15    function jugar() {
16      debugger;
17      var intentos = 5;   intentos = 5
18      var numerito = Math.floor(Math.random() * 100 + 1);   numerito = 3
19      console.log(numerito);
20      var encontrado = false;
21      for (let i = 1; i <= intentos && !encontrado; i++) {
22        let num = prompt("Tienes 5 intentos. Intento " + i + ". \n In
23        if (
```

```
ƒ prompt()
length: 0
name: "prompt"
arguments: (...)
caller: (...)
▸ [[Prototype]]: ƒ ()
▸ [[Scopes]]: Scopes[0]
```

Valor asignado a la variable

Instrucción en ejecución

Haciendo ***clic*** en el número de línea se establece un ***punto de interrupción***

Pasando el cursor por una función, arroja información

(F8) ***Reanuda*** la ejecución ***hasta*** el siguiente punto de ***interrupción*** si lo hay

(F10) ***Step Over***

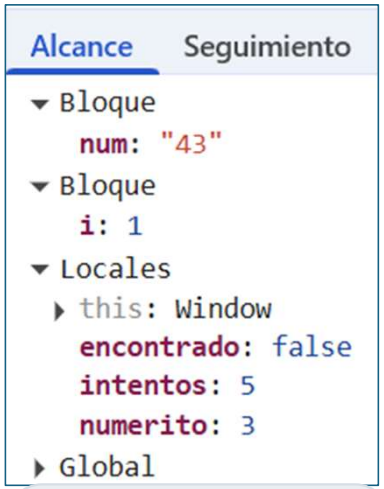
Pasa a la siguiente llamada de ***función***

Salir de la función actual

(F9) ***Step Into***

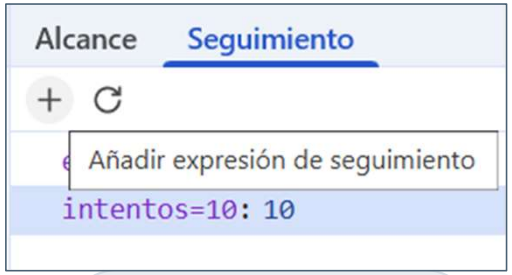
Siguiente instrucción

(Ctrl-F8) ***Desactiva*** puntos de interrupción.



Alcance (Scope):

Podemos ver variables ***visibles***, su ***valor*** y su ***alcance***



Seguimiento (Watch):

+ Podemos monitorizar variables.

Doble click en la variable nos permite cambiar el valor.

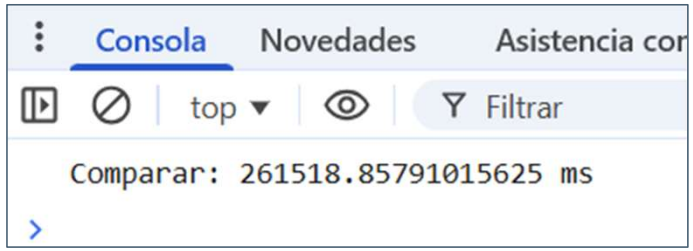
Pila de llamadas (Call stack):
Podemos ver en qué función no encontramos, útil en el caso de funciones anidadas.
Anónimo (Chrome) o Global (FireFox) => función ***principal***

```javascript
// Comprobamos si todas las letras de word1 están en word2
console.time("Comparar");
let flag = true;
set1.forEach(letra => { flag = flag && set2.has(letra);  });

// Si coinciden, comprobamos si todas las letras de word2 están en word1.
if (flag) {
  set2.forEach(letra => {flag = flag && set1.has(letra);  });
}
console.timeEnd("Comparar");
```

El navegador incorpora un método muy sencillo para medir el tiempo empleado en realizar una/s sentencia/s

console.time(“Comparar”) => Empieza a contar el tiempo

console.timeEnd(“Comparar”) =>

Detiene el cronómetro y muestra en consola el valor en ms